



eLektrijada

Objektni model

Verzija: 2.0

Autori: Andrej Tomić, Marko Dunović, Danilo Todorović, Damjan Štrbac, Dragan Čolić

Datum: 15.1.2026.

1	Uvod	3
2	Domenski dijagram klasa	4
3	Frontend.....	5
3.1	Inicijalizacija sistema	5
3.2	Prijava i registracija.....	7
3.3	Prikaz prijavljenih problema	9
3.4	Pregled rasporeda takmičenja.....	11
3.5	Pregled novosti.....	12
3.6	Pregled rezultata takmičenja	13
3.7	Pregled timova.....	14
4	Backend	15
4.1	Administratorski i korisnički nalog.....	15
4.2	Prijava problema	16
4.3	Rezultati takmičenja.....	18
4.4	Raspored takmičenja	19
4.5	Novosti	20

1 Uvod

U ovom dokumentu je predstavljen objektni model za eLektrijada softverski sistem upotrebom UML dijagrama klasa. Dokument je podijeljen u tri dijela: domenski dijagram klasa, frontend i backend.

Front-end dio predstavlja komunikaciju na strani korisnika (mobilna aplikacija) i slanje podataka na mrežu prema serveru. Aplikacija je realizovana kao varijacija **MVVM (Model-ViewViewModel)** arhitekture sa sljedećim razlikama u organizaciji:

- **ViewModel**

U ovoj aplikaciji je predstavljen kroz ekrane i komponente. Ove komponente obavljaju poslovnu logiku i interakciju sa korisnikom. Na primjer, komponente koriste **useEffect**, **useState** i **useNavigation** za upravljanje stanjima, pozivanje funkcija i navođenje odgovarajućih akcija, kao što su navigacija između ekrana. Sve funkcije koje se tiču interakcije s korisnikom i manipulacije podacima predstavljaju **ViewModel**.

- **View**

View u ovoj aplikaciji je striktno odvojen od poslovne logike i obuhvata samo stilizovanje i prikazivanje korisničkog interfejsa. To uključuje **CSS** stilove koji definišu izgled elemenata na ekranu. Takođe, **JSX** je dio view-a, jer se koristi za strukturisanje HTML-a i organizovanje korisničkog interfejsa u aplikaciji.

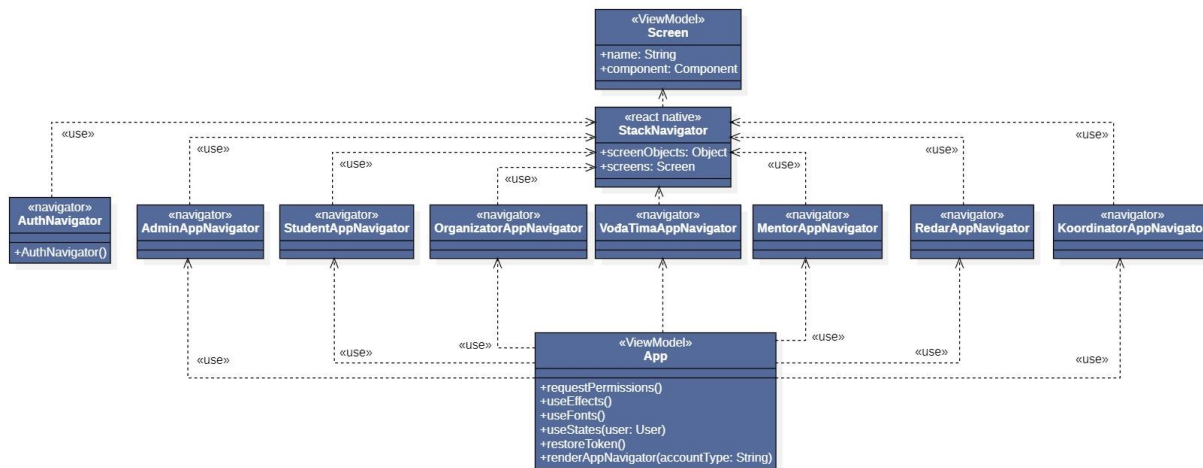
Na ovaj način, **ViewModel** (ekrani i komponente) komunicira sa korisnikom i obrađuje logiku, dok **View** (CSS i JSX) obavlja isključivo prikazivanje podataka i stilizovanje interfejsa. Ovaj pristup omogućava jasnu podjelu odgovornosti. Neki od ekrana direktno obrađuju podatke koje dobijaju preko **HttpService** klase koja služi samo za proslijeđivanje zahtjeva backend-u aplikacije.

Backend dio predstavlja komunikaciju na serverskoj strani sa bazom podataka.

Tabela zaduženja			
Član tima	Zaduženje		
Marko Dunović	3.1	3.2	4.1
Andrej Tomić	3.3	4.2	
Danilo Todorović	3.4	4.4	
Dragan Čolić	3.5	4.5	
Damjan Štrbac	3.6	3.7	4.3

3 Frontend

3.1 Inicijalizacija sistema



Prikazani dijagram predstavlja arhitekturu korisničkog interfejsa aplikacije razvijene u React Native okruženju, sa jasnim fokusom na navigaciju prilagođenu različitim tipovima korisnika. Aplikacija je strukturisana tako da koristi StackNavigator komponentu za upravljanje ekranima, a glavna logika aplikacije se nalazi u klasi App, koja je zadužena za inicijalizaciju, logiku autentifikacije i prikaz odgovarajućih navigatora u zavisnosti od tipa korisnika.

1. Inicijalizacija aplikacije – klasa App

Klasa App predstavlja ulaznu tačku aplikacije. Njen zadatak je da:

- Pozove metode kao što su requestPermissions() za traženje dozvola (npr. notifikacije),
- Inicijalizuje fontove, efekte i stanje korisnika pomoću useFonts(), useEffects() i useStates(),
- Pronađe autentifikacioni token putem restoreToken(),
- Na osnovu tipa naloga (prosleđenog kroz accountType) prikaže odgovarajući navigator koristeći renderAppNavigator(accountType: String).

2. Navigacija prema tipu korisnika

Na osnovu korisničkog naloga, aplikacija prikazuje odgovarajući navigator. Svaki navigator predstavlja skup ruta specifičnih za jednu grupu korisnika:

- AdminNavigator
- StudentNavigator
- OrganizatorNavigator
- VođaTimaNavigator
- MentorNavigator
- RedarNavigator
- KoordinatorNavigator

Svi oni koriste StackNavigator za organizaciju i upravljanje ekranima (Screen). Svaki ekran (Screen) je definisan imenom i pripadajućom React komponentom, što omogućava fleksibilnu i jednostavnu konfiguraciju prikaza.

Ako korisnik nije prijavljen ili nije poznat njegov tip, koristi se AuthNavigator za prikaz početnih ekrana kao što su:

- Ekran za prijavu
- Registracija
- Oporavak lozinke
- Pregled novosti i dešavanja

3. StackNavigator komponenta

StackNavigator je centralna komponenta koja omogućava navigaciju između ekrana u okviru svakog navigatora. Ona održava stek ekrana, podržava navigaciju naprijed/nazad, i koristi:

- screenObjects: objekt koji mapira imena na komponente
- screens: konkretni Screen objekti s definisanim atributima name i component

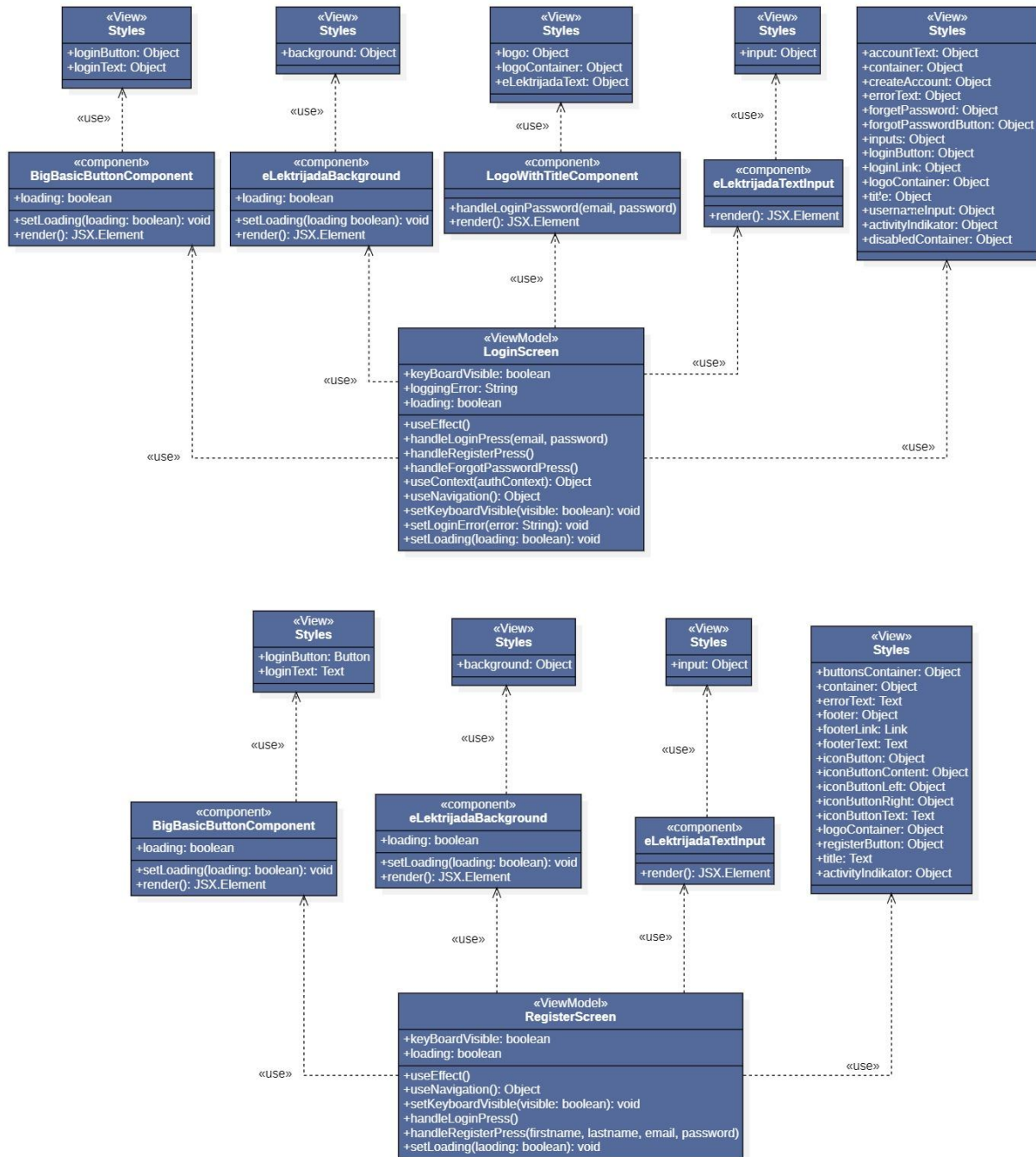
4. Modularna i skalabilna struktura

Zahvaljujući ovom dizajnu, aplikacija omogućava jasno razgraničenje funkcionalnosti po tipu korisnika. Svaka korisnička grupa ima svoj navigator koji sadrži rute i ekrane relevantne samo za njih, čime se postiže:

- Bolja organizacija koda
- Jednostavnije održavanje
- Laka proširivost funkcionalnosti

Ovaj dijagram efikasno prikazuje kako aplikacija koristi princip razdvajanja odgovornosti i komponentni pristup, što je u skladu s najboljim praksama razvoja u React Native okruženju.

3.2 Prijava i registracija



U okviru aplikacije eLektrijada, korisnički interfejs za autentifikaciju korisnika organizovan je kroz jasno strukturirane ekrane i komponente korišćenjem React Native tehnologije. Klase LoginScreen i RegisterScreen implementirane su kao ekrani (screen klase), koje predstavljaju samostalne funkcionalne cjeline za prijavu i registraciju korisnika. Njihova funkcionalnost i izgled zasnivaju se na ponovnom korišćenju specijalizovanih komponenti, čime se postiže modularnost i efikasnost prilikom razvoja korisničkog interfejsa.

Konkretno, LoginScreen i RegisterScreen sastavljeni su od sledećih React komponenti:

- BigBasicButtonComponent – služi za prikaz osnovnog dugmeta za potvrdu akcija (npr. prijava, registracija), uz podršku za vizuelnu indikaciju učitavanja (loading).
- eLektrijadaBackground – komponenta zadužena za prikaz pozadinske grafike ili dizajna specifičnog za aplikaciju Elektrijada.
- eLektrijadaTextInput – prilagođena ulazna komponenta za unos podataka, poput korisničkog imena ili lozinke.
- LogoWithTitleComponent – prikazuje logo aplikacije i odgovarajući naslov, uz integrisanu logiku za obradu prijave putem metode handleLoginPress.

Ove komponente zajedno čine ViewModel sloj, jer osim prikaza sadrže i funkcionalnosti neophodne za ponašanje ekrana – kao što su obrade događaja (klikova, unosa, grešaka) i promjene stanja (loading, keyboardVisible, loginError). Na primjer, klasa LoginScreen uključuje metodu handleLoginPress, koja se poziva pri pokušaju prijave i koristi podatke unesene u eLektrijadaTextInput.

Prikaz korisničkog interfejsa dodatno je podržan klasama Styles, koje predstavljaju View sloj. Svaka komponenta koristi zaseban Styles objekat koji definiše izgled elemenata kroz stilove poput input, loginButton, logo, errorText, activityIndicator i drugih. Na ovaj način, omogućava se fleksibilna vizuelna prezentacija svakog dijela interfejsa, uz zadržavanje funkcionalne logike odvojene od prikaza.

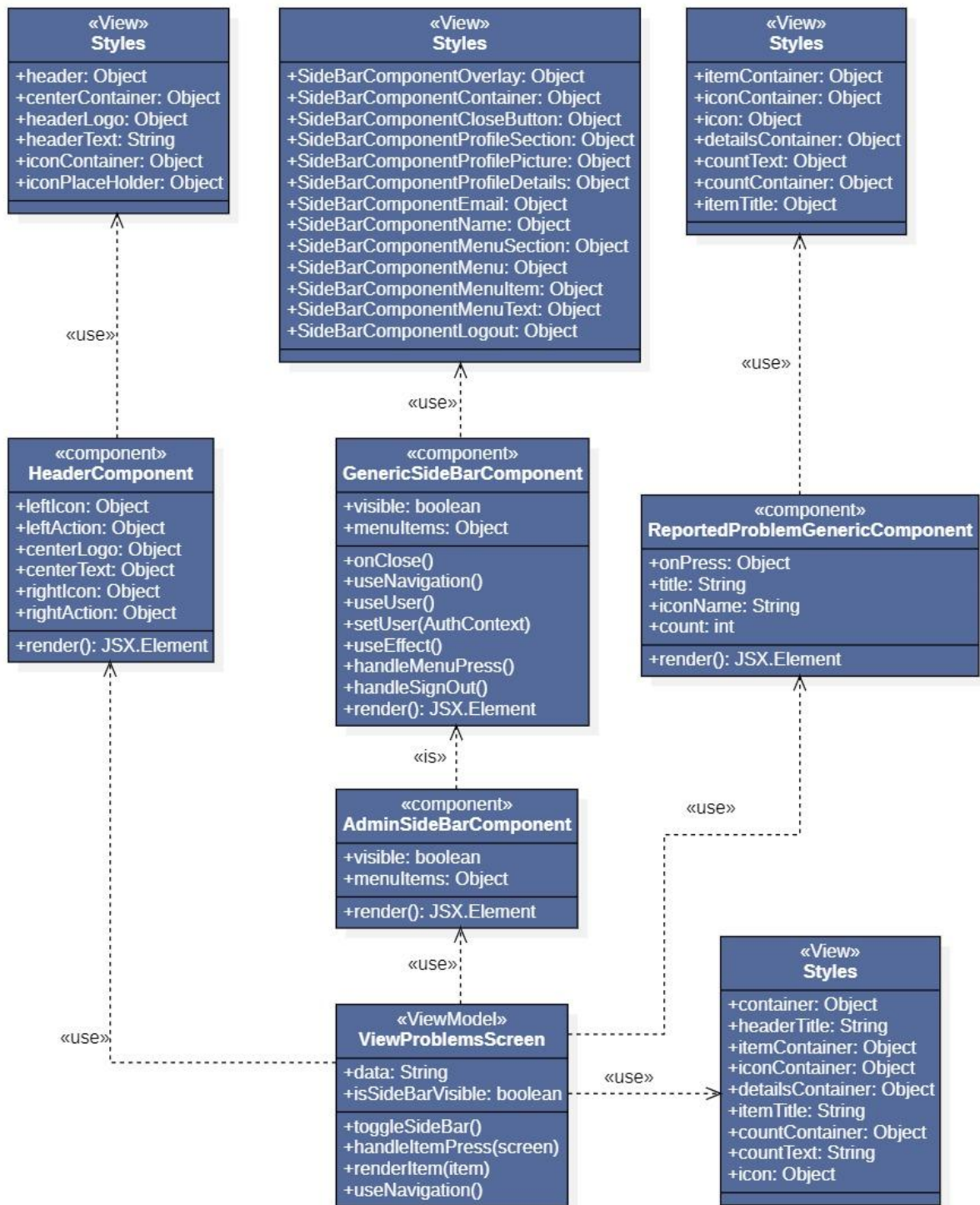
Korisnički tok započinje prikazom LoginScreen kao inicijalnog ekrana aplikacije. Sa ovog mjesta, korisnik može direktno da pređe na RegisterScreen radi kreiranja novog naloga, ili da aktivira proceduru za oporavak lozinke kroz poseban ekran ResetPasswordScreen. Time se obezbeđuje jednostavna i intuitivna navigacija unutar autentifikacijskog dijela aplikacije eLektrijada.

Ovakav arhitektonski pristup omogućava:

- ponovnu upotrebu komponenti,
- bolju testabilnost i održavanje,
- jasno razdvajanje logike i prikaza,

što čini sistem fleksibilnim i spremnim za dalje proširenje.

3.3 Prikaz prijavljenih problema



Prikazani dijagram klasa odnosi se na frontend aplikacije koja omogućava administratoru pregled i upravljanje prijavljenim problemima. Administrator ima pristup različitim sekcijama u kojima može da pregleda prijavljene probleme koje ostavljaju korisnici. Svaka od ovih

funkcionalnosti implementirana je kroz posebne komponente i stilove radi bolje organizacije i modularnosti.

Dijagram sadrži više ključnih komponenti:

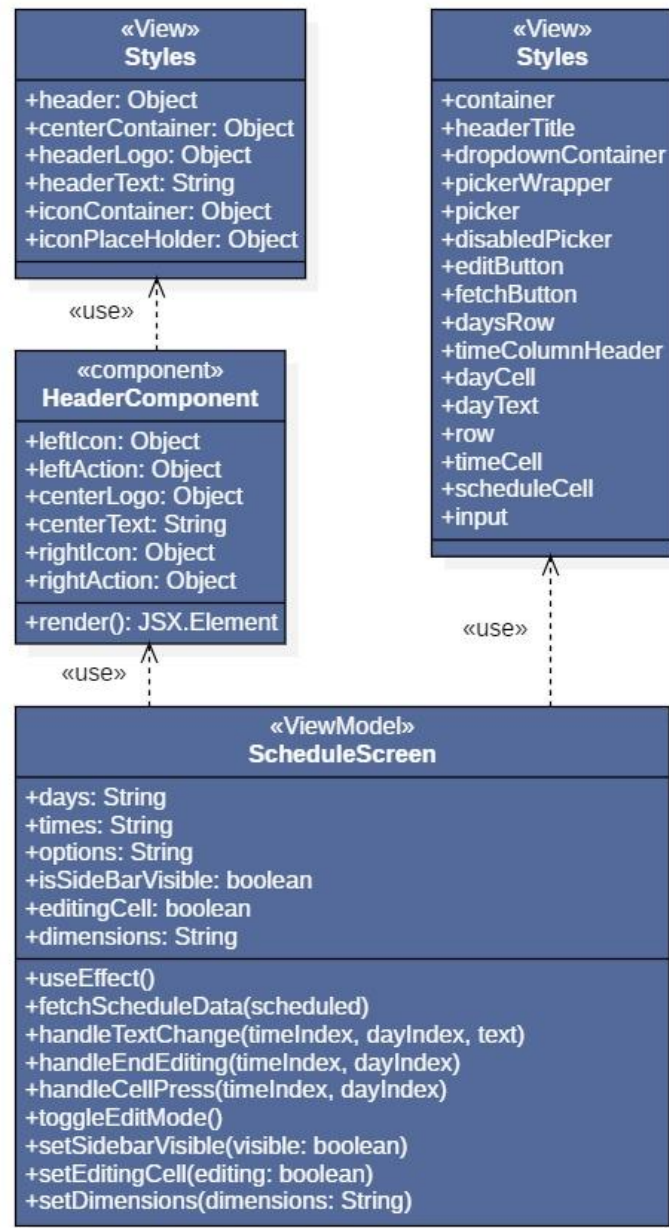
- AdminSideBarComponent je specijalizacija generičke komponente GenericSideBarComponent, i koristi se za prikaz navigacije u obliku bočne trake (SideBar) kroz koju administrator prelazi između ekrana.
- Komponenta ViewProblemsScreen predstavlja ViewModel koji upravlja prikazom liste problema i kontrolom prikaza bočne trake. Sadrži metode za navigaciju, prikaz pojedinačnih stavki i njihovu obradu.
- Komponenta ReportedProblemGenericComponent koristi se za prikaz pojedinačnih prijava problema i omogućava interakciju korisnika putem događaja onPress.

Pored toga, sve komponente koriste odgovarajuće stilove koji su organizovani kroz posebne klase Styles. Ove klase definišu izgled zaglavlja (HeaderComponent), bočne trake (GenericSideBarComponent i AdminSideBarComponent) i kartica za prikaz problema (ReportedProblemGenericComponent).

Stilovi su razdvojeni prema tipu komponente i omogućavaju lako održavanje i prilagođavanje korisničkog interfejsa. Svaka komponenta ima jasno definisane elemente koje koristi, kao i metode koje omogućavaju dinamičko ponašanje aplikacije, poput promjene korisnika, zatvaranja menija i navigacije.

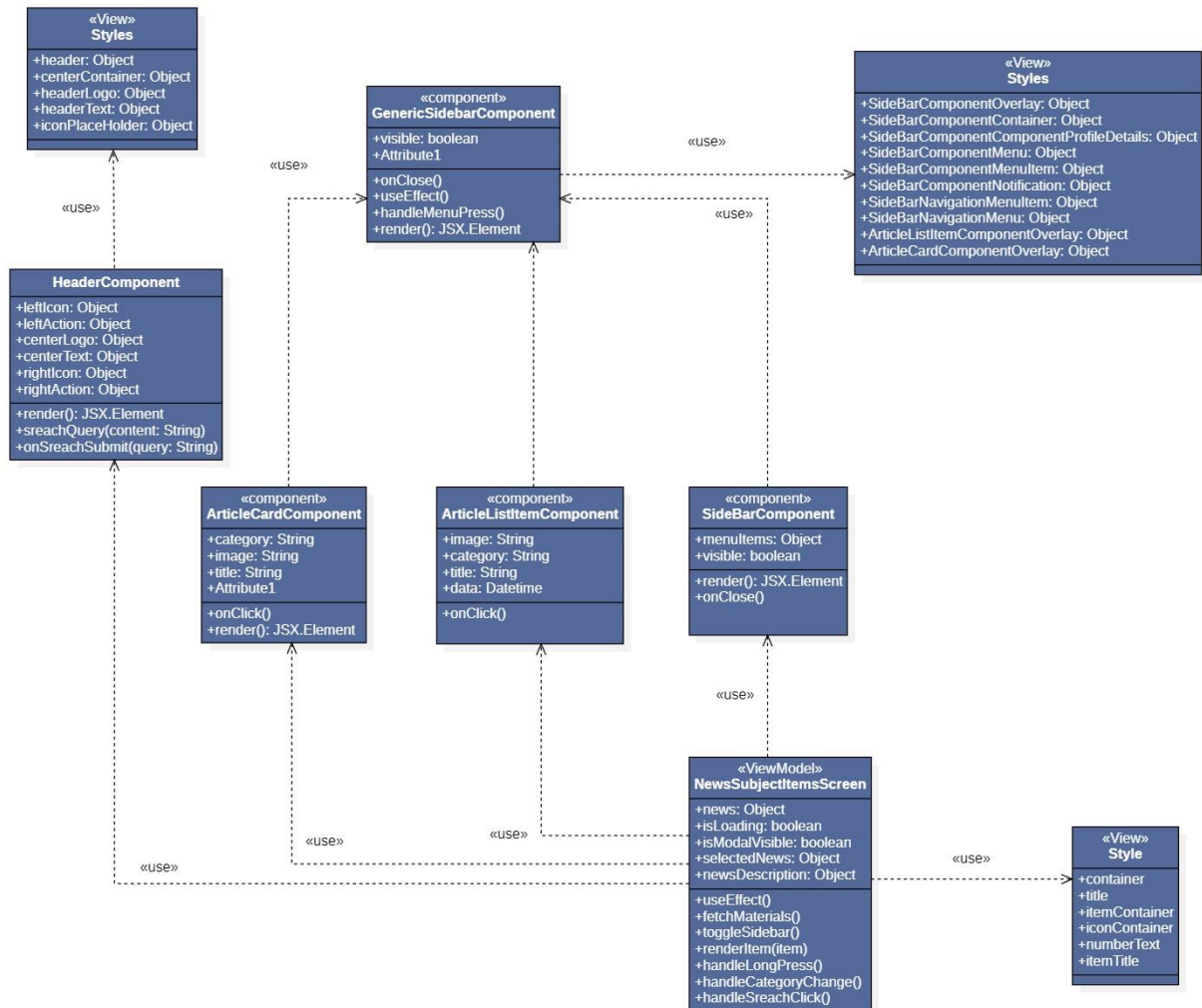
Na ovaj način je omogućeno jasno odvajanje logike prikaza, ponašanja i stilova, što doprinosi boljoj skalabilnosti i preglednosti frontenda aplikacije.

3.4 Pregled rasporeda takmičenja



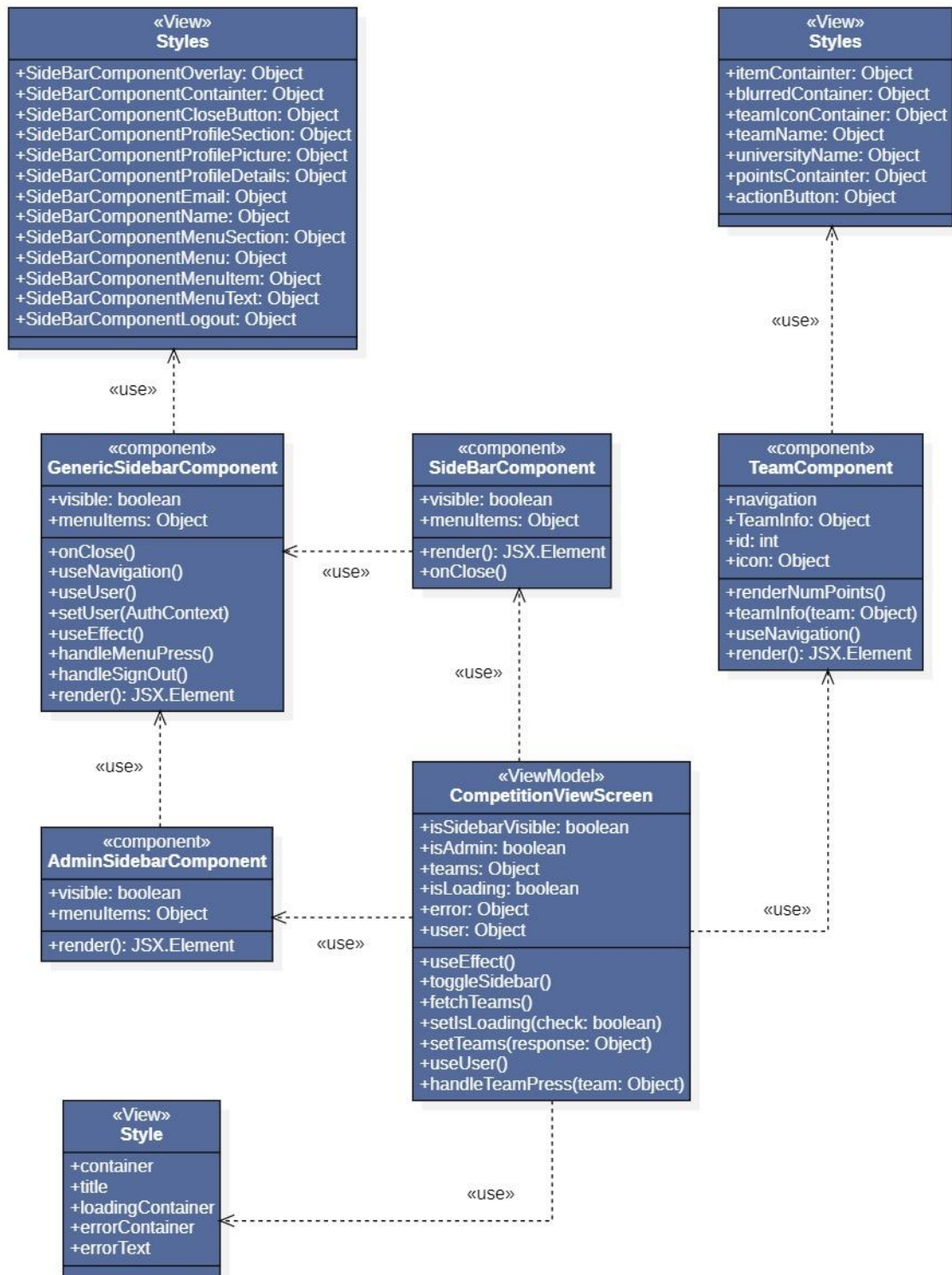
Dijagram prikazuje pojednostavljenu strukturu prikaza rasporeda u aplikaciji. Glavna logika prikaza nalazi se unutar ScheduleScreen klase, koja predstavlja osnovni prikaz i upravlja dohvatanjem i prikazivanjem podataka o rasporedu. Klasa ScheduleScreen komunicira sa komponentom HeaderComponent, koja služi za prikaz fiksnog zaglavlja aplikacije vidljivog na svim ekranima. Stilizacija elemenata se odvija kroz klasu Styles, gdje su definisani stilovi za prikaz rasporeda, dana, vremena i drugih elemenata tabele. Ovakva struktura omogućava jednostavno korišćenje i preglednost na svim uređajima, uz minimalnu kompleksnost koda.

3.5 Pregled novosti



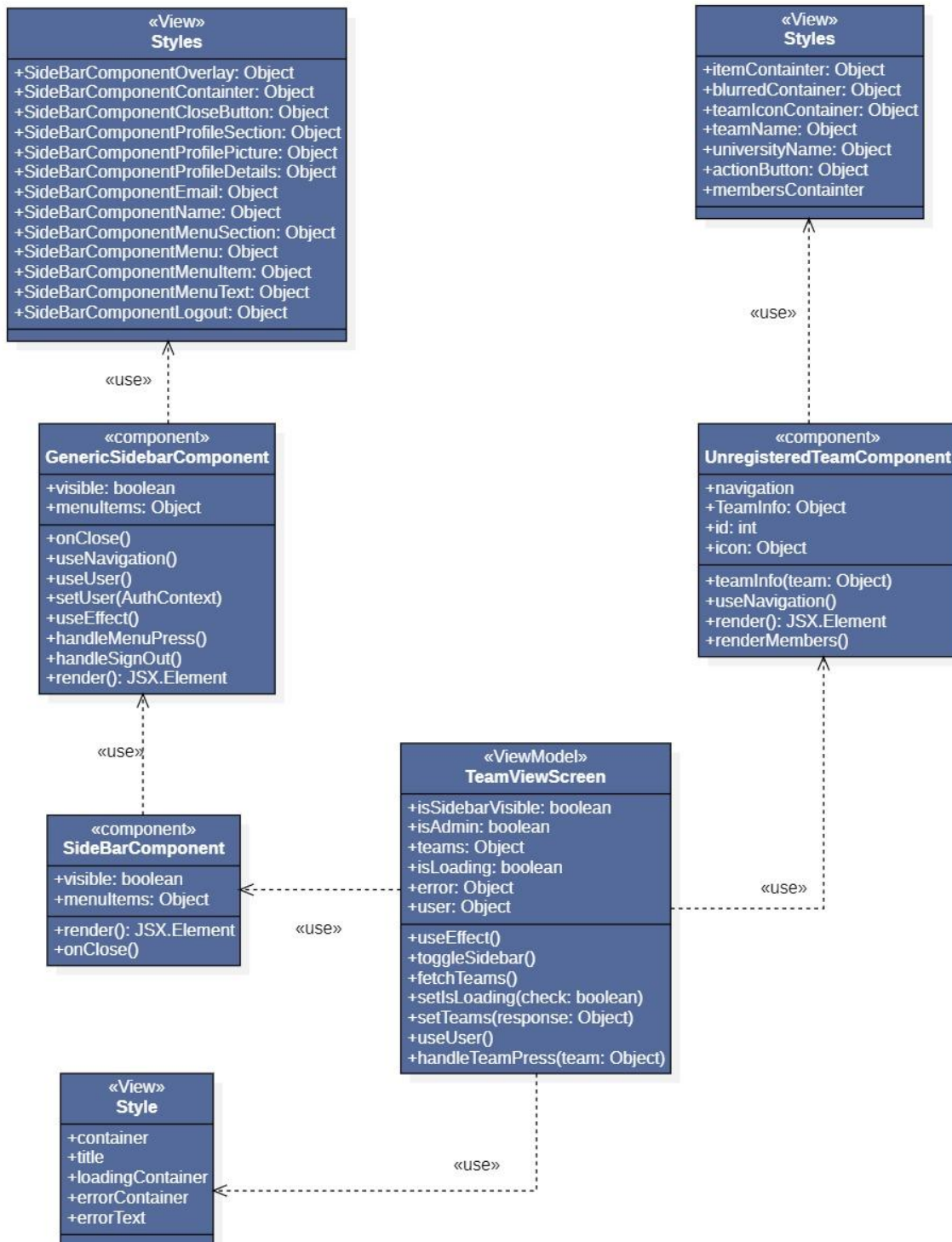
Dijagram klasa prikazuje strukturu i funkcionalnosti dijela aplikacije koji se odnosi na pregled novosti. Pošto je ovaj dio jedan od glavnih ekrana, sadrži Sidebar koji je modularan, sa generičkim, administratorskim i specifičnim implementacijama, omogućavajući prikaz elemenata menija i interakciju korisnika. Ekran za pregled novosti koristi podatke o novostima i omogućava akcije poput prikaza, izmjene i brisanja stavki. Stilovi su posebno definisani za svaki element radi bolje organizacije i ponovne upotrebe. Novosti su raspoređene po kategorijama i postoji dodatno polje za pretragu (sreach). Novosti mogu da dodaju organizatori, a njihov sadržaj može da pregleda bilo koji korisnik. Administrator može da uklanja novosti. Važno je napomenuti da u zavisnosti od role korisnika aplikacije se prikazuju različiti sadržaji mogućnosti izvršavanja akcija.

3.6 Pregled rezultata takmičenja



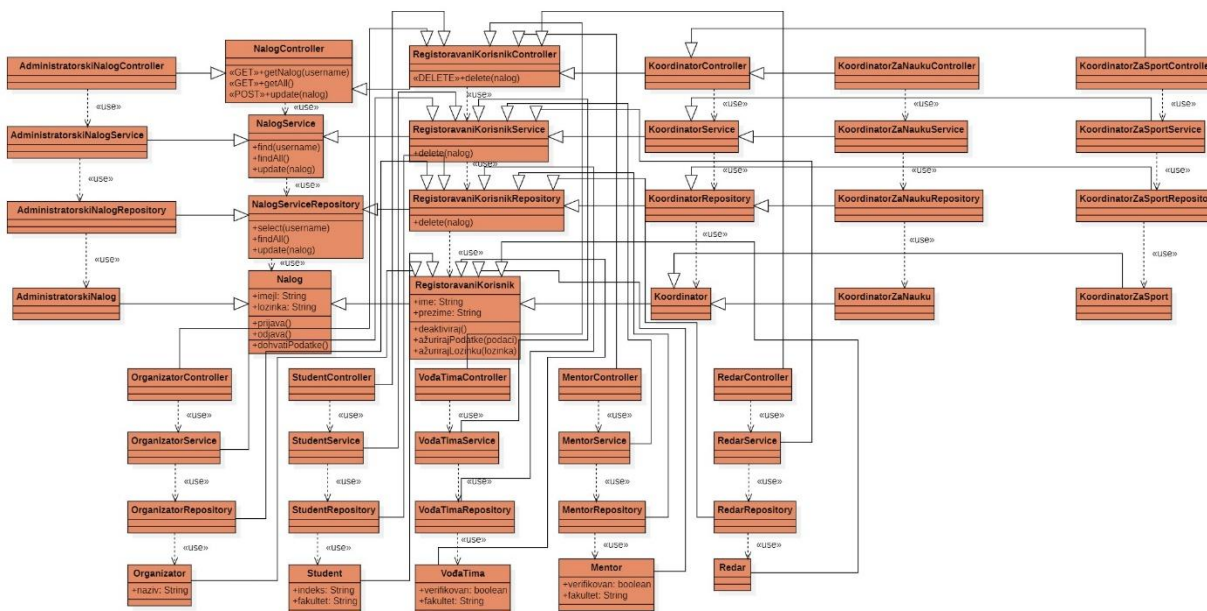
Neophodna komponenta u smislu funkcionalnosti je prikaz rezultata takmičenja. Ove rezultate mogu pregledati svi korisnici sistema. Ekran pregled takmičenja se sastoji od više komponenta (TeamComponent) u kojoj imamo više informacija o samoj ponudi.

3.7 Pregled timova



Ovaj dijagram prikazuje arhitekturu prikaza timova iz perspektive vođe tima u aplikaciji, koristeći komponente i modele iz React Native okruženja. Centralnu ulogu ima `TeamViewScreen`, koji predstavlja glavni ViewModel i sadrži podatke o korisniku, timovima, stanju učitavanja, greškama i vidljivosti bočne trake.

4.1 Administratorski i korisnički nalog



Centralna apstraktna klasa je Nalog, koja sadrži osnovne atribute kao što su ime i lozinka, kao i metode za prijavu, odjavu i dohvat podataka. Na osnovu nje izvedene su klase AdministratorskiNalog, RegistrovaniKorisnik, Koordinator, i ostale specijalizovane podklase.

- AdministratorskiNalogController
- AdministratorskiNalogService
- AdministratorskiNalogRepository

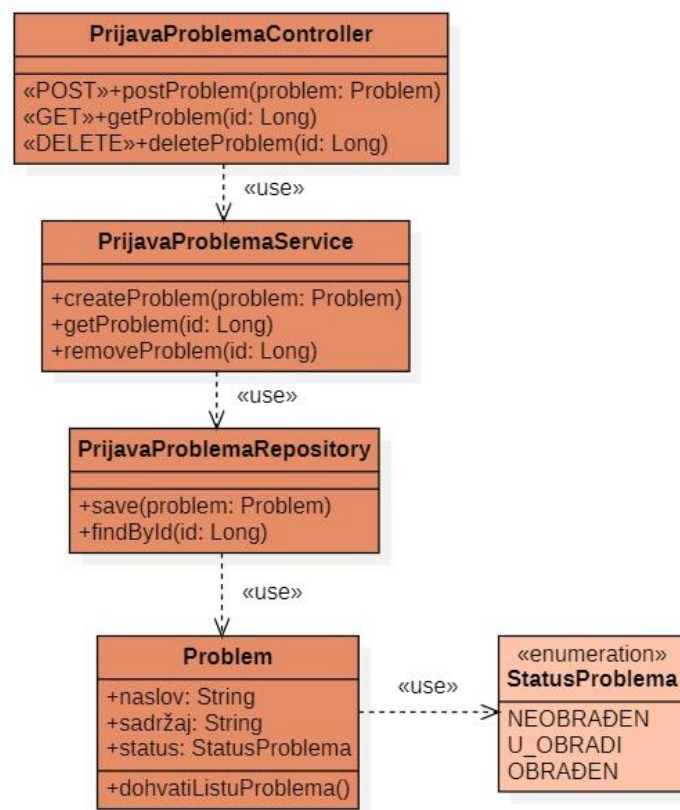
- Student, Organizator, Mentor, Redar, Koordinator, VodaTima, koji svi nasljeđuju ili proširuju RegistrovaniKorisnik ili Koordinator

- Svaki od njih ima svoje odgovarajuće Controller–Service–Repository klase, npr. StudentController, StudentService, StudentRepository.

Posebna pažnja je posvećena koordinatorskim nalogima. Klasa Koordinator se dalje dijeli na KoordinatorZaNauku i KoordinatorZaSport, sa posebnim slojevima Controller–Service–Repository za svaku od ovih funkcija, omogućavajući jasno razgraničenje nadležnosti i odgovornosti u sistemu.

Ukratko, ovaj dijagram jasno oslikava modularni pristup i princip odvajanja odgovornosti, što je karakteristično za Spring Boot aplikacije koje koriste JPA. Kroz ovakav dizajn omogućeno je lako upravljanje i proširivanje sistema u skladu s potrebama aplikacije.

4.2 Prijava problema



Dati dijagram klasa prikazuje backend strukturu modula za prijavu problema, koji omogućava komunikaciju servera sa bazom podataka radi čuvanja i upravljanja informacijama o prijavljenim problemima. Dijagram je izrađen prema principima SpringBoot arhitekture u kombinaciji sa JPA (Java Persistence API) pristupom.

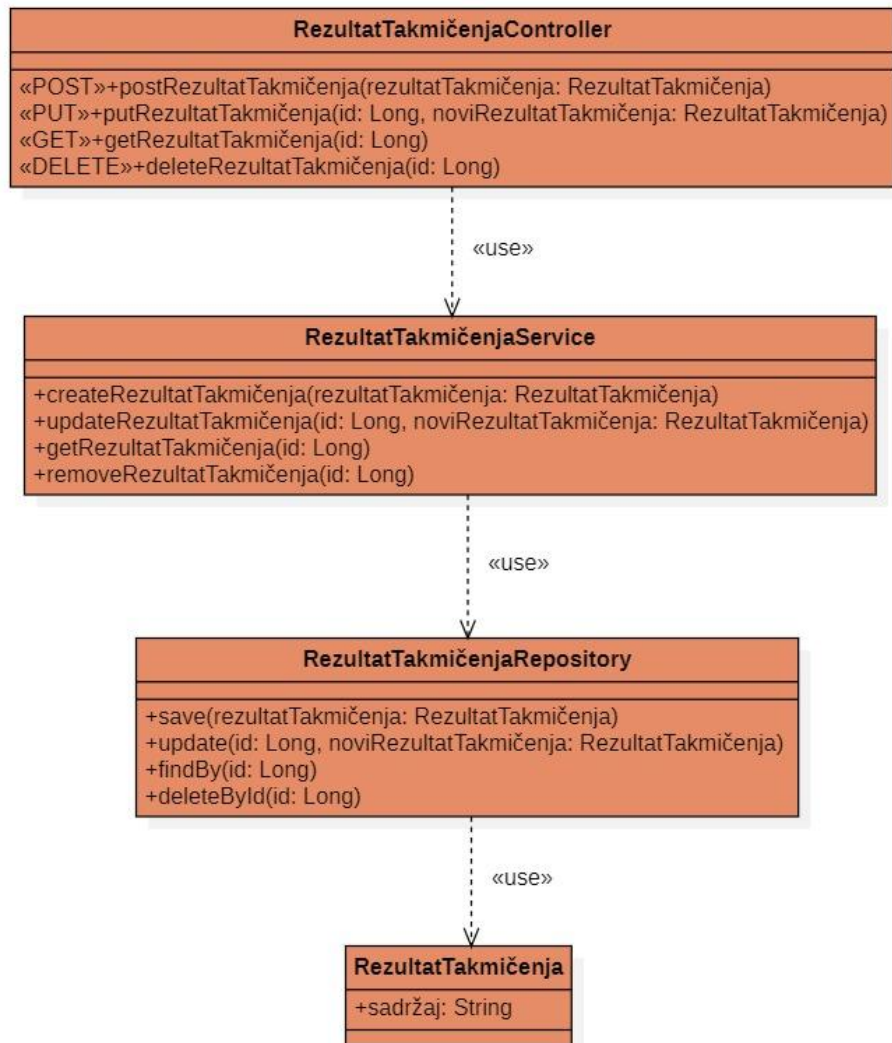
Za domensku klasu **Problem**, koja sadrži atribut `naslov`, `sadržaj` i `status`, definisane su odgovarajuće klase: **PrijavaProblemaController**, **PrijavaProblemaService** i **PrijavaProblemaRepository**, koje predstavljaju tipičan trostruki sloj arhitekture (Controller - Service - Repository).

- `PrijavaProblemaController` sadrži REST metode za kreiranje (POST), dobavljanje (GET) i brisanje (DELETE) problema prema njihovom ID-ju.
- `PrijavaProblemaService` implementira poslovnu logiku kroz metode za kreiranje, dohvaćanje i uklanjanje problema.
- `PrijavaProblemaRepository` komunicira direktno sa bazom podataka, koristeći standardne JPA metode poput `save` i `findById`.

Klasa `Problem` koristi enumeraciju `StatusProblema`, koja određuje status problema kroz unaprijed definisane vrijednosti: `NEOBRAĐEN`, `U_OBRADI` i `OBRAĐEN`.

Dijagram jasno ilustruje odvojenost slojeva u arhitekturi, modularnost i povezanost između kontrolera, servisa i repozitorijuma, što omogućava laku održivost i proširivost sistema.

4.3 Rezultati takmičenja



Dati dijagram klasa predstavlja komunikaciju sa bazom podataka radi pristupa podacima rezultata takmičenja. Na osnovu strukture SpringBoot komunikacije zajedno sa JPA, koristimo odgovarajući Controller, Service i Repository.

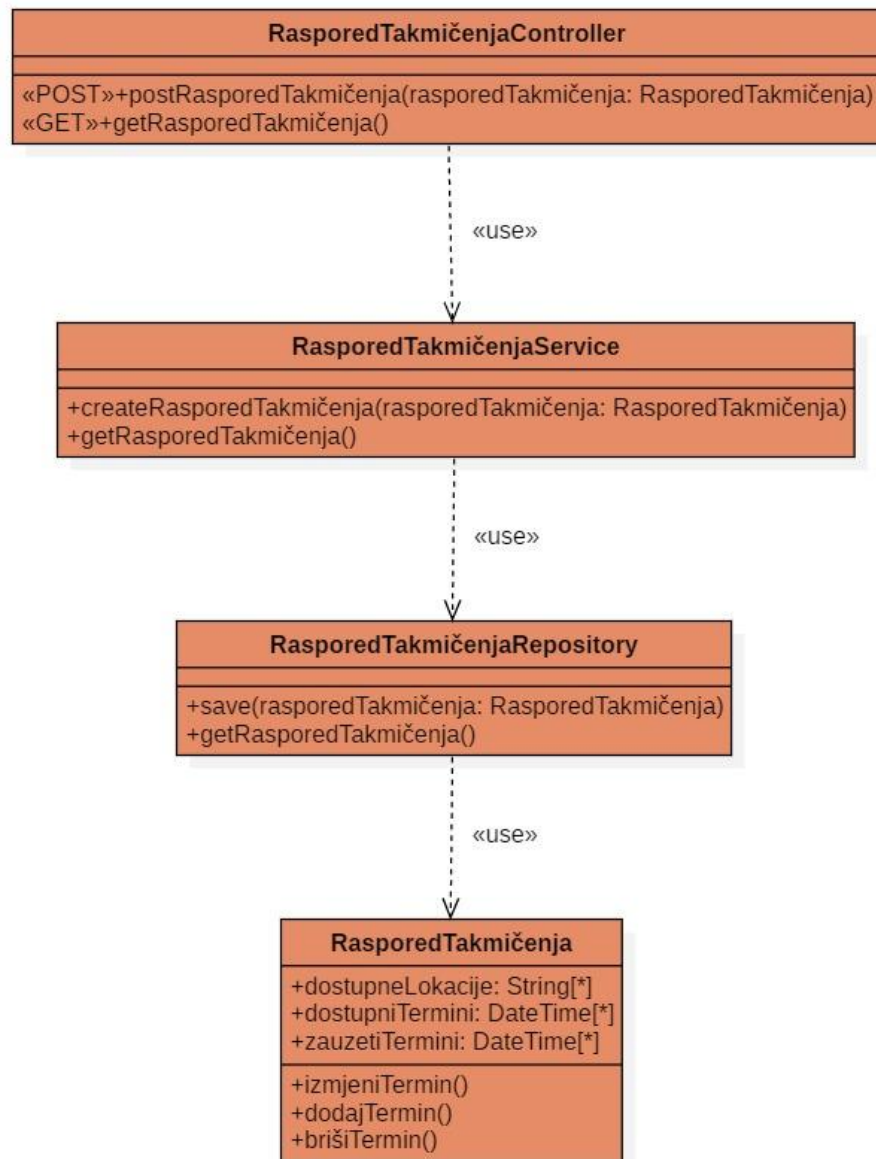
RezultatTakmicenjaController ima metode za dobijanje, kreiranje, ažuriranje i brisanje pojedinačnih rezultata takmičenja.

RezultatTakmicenjaService upravlja poslovnom logikom sa metodama za dobijanje, kreiranje i ažuriranje rezultata takmičenja.

RezultatTakmicenjaRepository sadrži metode za pronalaženje, čuvanje, ažuriranje i brisanje rezultata takmičenja u bazi podataka.

Ovaj sistem omogućava sveobuhvatno upravljanje porukama i grupama korisnika, pružajući potrebne CRUD operacije (Create, Read, Update, Delete) putem kontrolera, servisa i repozitorijuma.

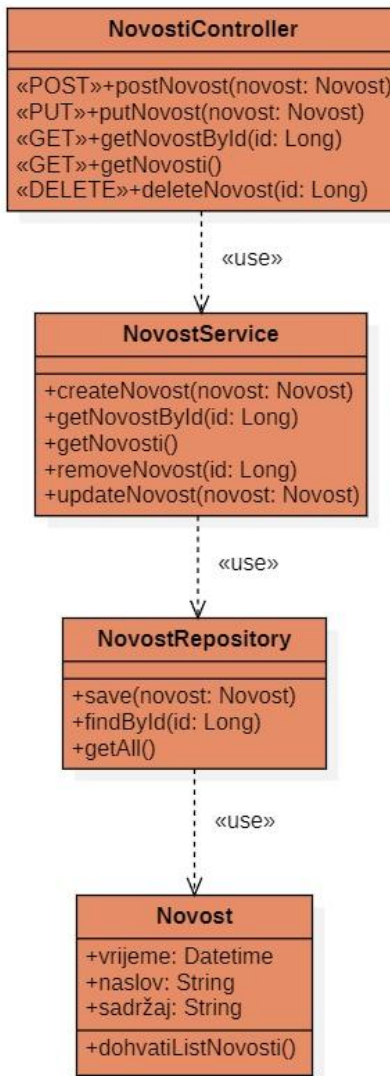
4.4 Raspored takmičenja



Dati dijagram klasa prikazuje komunikaciju servera sa bazom podataka u cilju pristupa informacijama o rasporedu takmičenja. Na osnovu strukture SpringBoot aplikacije u kombinaciji sa JPA tehnologijom, za svaku domensku klasu (u ovom slučaju RasporedTakmičenja) implementirane su odgovarajuće komponente: Controller, Service i Repository.

Na ovaj način, sistem omogućava jednostavno upravljanje rasporedom takmičenja, dok je slojevita arhitektura u skladu sa principima dobre prakse razvoja web aplikacija, omogućavajući lakšu održivost, testiranje i proširivost koda.

4.5 Novosti



Dati dijagram klasa predstavlja tipičnu arhitekturu slojeva u Spring Boot aplikaciji za upravljanje podacima o novostima. Na osnovu strukture Spring Boot komunikacije zajedno sa JPA (Java Persistence API), za svaku domensku klasu (u ovom slučaju Novost) imamo odgovarajući Controller, Service i Repository. Na vrhu hijerarhije se nalazi klasa NovostiController, koja predstavlja RESTful web servis za manipulaciju novostima. NovostiController ima metode namenjene za kreiranje nove, ažuriranje, dohvaćanje jedne, dohvaćanje svih i brisanje novosti. Sledeća u lancu je klasa NovostService koja sadrži metode za kreiranje nove novosti, dohvaćanje novost po ID-u, za dohvaćanje svih novosti, uklanjanje novosti, ažuriranje novosti. Na kraju, klasa Novost predstavlja sam domenski entitet u aplikaciji.